```
function [retmap,retvisited,retsteps] = dijkstras(mapfile,startlocation,targetlocation)

% Convert the map file into a MATLAB matrix
map = map_convert(mapfile);
retmap = map;

% Get dimensions of the map
[maprows,mapcols] = size(map);

% Distance from the start to every cell
distance = inf(maprows,mapcols);

% Keep track of whether a cell has been visited
visited = false(maprows,mapcols);

% Keep track of the parent of every cell
parent = zeros(maprows,mapcols,2);

% Distance from start to itself is zero
distance(startlocation(1),startlocation(2)) = 0;

% Store the order in which cells are visited
retvisited = [];

% Directions:
%       up
%       down
%       left
%       right
directions = [
    -1  0
     1  0
     0 -1
     0  1
];

% Dijkstra's algorithm
while true

    % Find the unvisited cell with the smallest distance
    smallestDistance = inf;
    current = [];

    for i = 1:maprows
        for j = 1:mapcols

            if ~visited(i,j) && map(i,j) == 0

                if distance(i,j) < smallestDistance

                    smallestDistance = distance(i,j);
                    current = [i,j];

                end
            end
        end
    end

    % No more reachable cells
    if isempty(current)
        break;
```

```
    end

    % Mark current cell as visited
    visited(current(1),current(2)) = true;

    % Record that this cell was visited
    retvisited = [retvisited; current];

    % Stop when target is reached
    if current(1) == targetlocation(1) && ...
       current(2) == targetlocation(2)

        break;
    end

    % Check neighbouring cells
    for k = 1:4

        neighbour = current + directions(k,:);

        row = neighbour(1);
        col = neighbour(2);

        % Check that neighbour is inside the map
        if row >= 1 && row <= maprows && ...
           col >= 1 && col <= mapcols

            % Check that neighbour is not a wall
            if map(row,col) == 0

                % Check that neighbour has not been visited
                if ~visited(row,col)

                    % Moving to an adjacent cell costs 1
                    newDistance = distance(current(1),current(2)) + 1;

                    % If this is a shorter route, update it
                    if newDistance < distance(row,col)

                        distance(row,col) = newDistance;

                        % Remember where we came from
                        parent(row,col,1) = current(1);
                        parent(row,col,2) = current(2);

                    end
                end
            end
        end
    end
end

% Reconstruct the shortest path
retsteps = [];

% Check whether target was reached
if distance(targetlocation(1),targetlocation(2)) ~= inf

    current = targetlocation;

    while ~(current(1) == startlocation(1) && ...
```

```
            current(2) == startlocation(2))

        % Add current cell to the path
        retsteps = [current; retsteps];

        % Get the parent of current cell
        parentrow = parent(current(1),current(2),1);
        parentcol = parent(current(1),current(2),2);

        current = [parentrow,parentcol];

    end

    % Add the starting location
    retsteps = [startlocation; retsteps];

end

plotmap(retmap, retsteps)

end


function placestep(position,i)

% This function will plot a yellow rectangle
% and print a number in the rectangle.

position = [16-position(1) position(2)];

position = [position(2)+0.1 position(1)+0.1];

rectangle('Position',[position,0.8,0.8],'FaceColor','y');

c = sprintf('%d',i);

text(position(1)+0.2,position(2)+0.2,c,'FontSize',10);

end
```